



AUDIT REPORT

June 2026

For



PredictorTM

Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Medium Severity Issues	14
1. Deprecated Chainlink latestAnswer() Used Without Validity Checks in usdcEth()	14
2. Permit-consuming lift functions are grievable by front-running the permit	15
3. Relay refund marks fees as refunded even when the ETH delivery silently fails	16
Functional Tests	17
Threat Model	28
Automated Tests	42
Closing Summary & Disclaimer	43



Executive Summary

Project Name	The Truth Network Foundation
Protocol Type	Cross-chain bridge
Project URL	https://predictor.io/
Overview	Truth Network's's PredictorBridge smart contract enables users and relayers to bridge supported tokens between Ethereum and a T2 prediction market system. The protocol supports token lifts, lower claims, relayer-assisted operations, author management, Merkle root publication, EIP-712 author confirmations, ERC-2612 permit-based transfers, and relayer fee refunds through Uniswap V3.
Audit Scope	The scope of this Audit was to analyze the Truth Network Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Predictor-Foundation/predictor-bridge
Contracts in Scope	PredictorBridge.sol
Branch	main
Commit Hash	15a76e63fece7def57a759bf7aa08faccfed00b5
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	25th May 2026 to 29th May 2026
Updated Code Received	4th June 2026
Review 2	5th June 2026
Fixed In	https://github.com/Predictor-Foundation/predictor-bridge/pull/1/changes/3ca4fea26d61997dd379da139b517457ae8b39cb



Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	3 (100.00%)
Low	0 (0%)
Informational	0 (0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	3	0	0



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Deprecated Chainlink latestAnswer() Used Without Validity Checks in usdcEth()	Medium	Resolved
2	Permit-consuming lift functions are grievable by front-running the permit	Medium	Resolved
3	Relayer refund marks fees as refunded even when the ETH delivery silently fails	Medium	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Using throw

✓ Using inline assembly

✓ Style guide violation

✓ Unsafe type inference

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Medium Severity Issues

Deprecated Chainlink latestAnswer() Used Without Validity Checks in usdcEth()

Resolved

Path

PredictorBridge.sol

Function Name

`usdcEth()`

Description

`usdcEth()` fetches the USDC/ETH price using the deprecated Chainlink `latestAnswer()`. This function is deprecated in favour of `latestRoundData()` and does not expose round metadata, making staleness and completeness validation impossible. Three concrete failure modes exist:

```
price = uint256(ICChainlinkV3Aggregator(CHAINLINK_USDC_ETH_FEED).latestAnswer()) / 1e6;
```

Impact

A bad feed value freezes all relayer-sponsored lift, lower, and refund paths or silently misprices fees and the slippage protection floor in `refundRelayerCallback`. No attacker action is required.

Likelihood

Triggers on any Chainlink misbehavior, operator outage, paused aggregator, or feed deprecation.

Recommendation

Replace the deprecated `latestAnswer()` with `latestRoundData()` and update the `ICChainlinkV3Aggregator` interface accordingly. Validate the returned data by rejecting zero or negative answers, treating `updatedAt` of zero as an incomplete round, ensuring `answeredInRound` is not less than `roundId`, and enforcing a staleness threshold against `block.timestamp` set slightly above the Chainlink heartbeat for the configured feed.



Medium Severity Issues

Permit-consuming lift functions are grieveable by front-running the permit

Resolved

Path

PredictorBridge.sol

Function Name

`permitLift()`, `predictionMarketPermitLift()`, `relayerLift()`

Description

All three permit-based lift entry points call ERC-2612 `permit()` unconditionally before pulling the user's tokens. Because `(v, r, s)` sits in the pending transaction's calldata, anyone watching the mempool can copy it and call `token.permit()` directly first. ERC-2612 nonces are single-use, so when the user's bundled call replays the now-consumed permit it reverts.

Impact

Grieveable DoS of the permit and relayer-sponsored lift paths. For `permitLift` and `predictionMarketPermitLift` the victim can recover by calling plain lift / `predictionMarketLift` since the front-run permit actually sets the allowance, but every `relayerLift` attempt wastes the relayer's gas and breaks the gasless UX guarantee.

Likelihood

Any mempool watcher can trigger this on any of the three entry points; freely repeatable.

Recommendation

Wrap the permit call in try/catch so an already-consumed permit does not abort the lift



Medium Severity Issues

Relayer refund marks fees as refunded even when the ETH delivery silently fails

Resolved

Path

PredictorBridge.sol

Function Name

`_attemptRelayerRefund()`

Description

refundRelayerCallback swaps the relayer's accrued USDC fees to ETH and forwards them to the relayer, but the success of the ETH send is explicitly thrown away. Because the inner call therefore never reverts on a failed send, the outer try in `_attemptRelayerRefund` runs its success branch and resets `relayerBalance` to the "registered with zero fees" sentinel.

```
(bool success, ) = relayer.call{ value: ethAmount }('');  
assembly { pop(success) }
```

If the send fails (the relayer is a contract with no payable receive, or its receive hook runs out of gas), the USDC fees are gone already swapped to ETH the converted ETH stays stuck in the bridge with no recovery path, and `relayerBalance` is permanently reset. `LogRefundFailed` is never emitted on this path, so the loss is invisible to monitoring.

Impact

Permanent loss of the relayer's accrued USDC fees and silent stranding of the swapped ETH in the bridge whenever the refund send fails. The ledger says refunded while no ETH ever arrived, and the stuck ETH cannot be retrieved without a contract upgrade.

Likelihood

Triggers when the relayer address cannot accept ETH (contract relayer without payable receive, or out-of-gas in receive). EOAs are unaffected; relayers are owner-registered, which limits the surface but does not prevent the silent state divergence.

Recommendation

Propagate the send result so a failed send unwinds the swap atomically



Functional Tests

PredictorBridge

- ✓ constructor reverts if the Chainlink USDC/ETH feed address is zero
- ✓ constructor reverts if the Uniswap V3 USDC/WETH pool address is zero
- ✓ constructor reverts if the Chainalysis sanctions oracle address is zero
- ✓ constructor reverts if the USDC token address is zero
- ✓ constructor reverts if the USDT token address is zero
- ✓ constructor reverts if the WETH token address is zero
- ✓ constructor stores all six external addresses as immutables in implementation bytecode
- ✓ constructor calls `_disableInitializers` so the implementation can never be initialized directly
- ✓ `initialize` can only be called once across the proxy's lifetime
- ✓ `initialize` reverts if the author count is below `MIN_AUTHORS` (4)
- ✓ `initialize` reverts if the author count is above `MAX_AUTHORS` (255)
- ✓ `initialize` reverts if `t1PubKeysLHS` length does not match `t1Addresses` length
- ✓ `initialize` reverts if `t1PubKeysRHS` length does not match `t1Addresses` length
- ✓ `initialize` reverts if `t2PubKeys` length does not match `t1Addresses` length
- ✓ `initialize` reverts if any supplied `t1Address` is the zero address
- ✓ `initialize` reverts if a derived address from (LHS,RHS) does not match the given `t1Address`
- ✓ `initialize` reverts if the same `t1Address` is supplied twice (`T1AddressInUse`)
- ✓ `initialize` sets the owner via `Ownable2Step`
- ✓ `initialize` sets `nextAuthorId` to 1 before seeding authors
- ✓ `initialize` activates each seed author so they immediately count toward quorum
- ✓ `initialize` initialises `Pausable` in the unpaused state



- ✓ `whenNotPaused` blocks every state-changing entrypoint while the contract is paused
- ✓ `withinCallWindow` reverts when `block.timestamp` is strictly greater than expiry
- ✓ `withinCallWindow` allows a call exactly at `block.timestamp` equal to expiry
- ✓ `checkAddress` reverts `AddressBlocked` when the sanctions oracle marks the address sanctioned
- ✓ `checkAddress` passes when the oracle reports the address is not sanctioned
- ✓ `ReentrancyGuardTransient` prevents reentry through external token or callback calls
- ✓ `addAuthor` reverts when paused
- ✓ `addAuthor` reverts `WindowExpired` when `block.timestamp` is past expiry
- ✓ `addAuthor` reverts `InvalidT1Key` if `t1PubKey` length is not 64 bytes
- ✓ `addAuthor` reverts `InvalidT2Key` if `t2PubKey` is zero
- ✓ `addAuthor` reverts `AlreadyAdded` if the derived `t1Address` is currently a registered author
- ✓ `addAuthor` reverts `BadConfirmations` if the supplied signatures do not reach quorum
- ✓ `addAuthor` reverts `TxIdsUsed` if the `t2TxId` was previously consumed
- ✓ `addAuthor` reverts `CannotChangeT2Key` if a rejoining author supplies a different `t2PubKey`
- ✓ `addAuthor` reverts `TooManyAuthors` if `nextAuthorId` has already reached 256
- ✓ `addAuthor` reverts `T2KeyInUse` if a brand-new `t2PubKey` is already bound to another id
- ✓ `addAuthor` allocates a fresh id and bidirectional mappings for a brand-new author
- ✓ `addAuthor` re-sets the `isAuthor` bit for a previously removed author keeping their id
- ✓ `addAuthor` leaves the new author inactive until their first confirmation is observed
- ✓ `addAuthor` does not change `numActiveAuthors` on the admission step itself
- ✓ `addAuthor` consumes the `t2TxId` via `_storeT2TxId` for replay protection
- ✓ `addAuthor` emits `LogAuthorAdded` with `t1Address`, `t2PubKey` and `t2TxId`
- ✓ `removeAuthor` reverts when paused



- ✓ `removeAuthor` reverts `WindowExpired` when past expiry
- ✓ `removeAuthor` reverts `InvalidT1Key` if `t1PubKey` is not 64 bytes
- ✓ `removeAuthor` reverts `NotAnAuthor` if `t2PubKey` does not map to a current author
- ✓ `removeAuthor` reverts `BadConfirmations` on missing quorum
- ✓ `removeAuthor` reverts `NotEnoughAuthors` if removal would drop `numActiveAuthors` to `MIN_AUTHORS` or below
- ✓ `removeAuthor` reverts `TxIdsUsed` for a replayed `t2TxId`
- ✓ `removeAuthor` clears the `isAuthor` bit for the target id
- ✓ `removeAuthor` clears the active bit and decrements `numActiveAuthors` when the author was active
- ✓ `removeAuthor` preserves the id and the `t1↔t2` key bindings so the author can rejoin
- ✓ `removeAuthor` emits `LogAuthorRemoved` with `t1Address`, `t2PubKey` and `t2TxId`
- ✓ `publishRoot` reverts when paused
- ✓ `publishRoot` reverts `WindowExpired` when past expiry
- ✓ `publishRoot` reverts `RootHashIsUsed` if the root has already been published
- ✓ `publishRoot` reverts `BadConfirmations` on missing quorum
- ✓ `publishRoot` reverts `TxIdsUsed` for a replayed `t2TxId`
- ✓ `publishRoot` sets `isPublishedRootHash[rootHash]` to true on success
- ✓ `publishRoot` consumes the `t2TxId`
- ✓ `publishRoot` emits `LogRootPublished` with the `rootHash` and `t2TxId`
- ✓ `publishRoot` makes no external calls so no reentrancy guard is needed
- ✓ `pause` can only be called by the current owner
- ✓ `pause` reverts if the contract is already paused
- ✓ `unpause` can only be called by the current owner
- ✓ `unpause` reverts if the contract is not paused
- ✓ `renounceOwnership` always reverts with "Disabled" so ownership cannot be lost



- ✓ `acceptOwnership` is required to complete a two-step transfer (`Ownable2Step`)
- ✓ `_authorizeUpgrade` restricts UUPS upgrades to the owner only
- ✓ `receive` accepts ETH from any sender so WETH unwrapping during refunds works
- ✓ `lift` reverts when paused
- ✓ `lift` reverts `AddressBlocked` if `msg.sender` is sanctioned
- ✓ `lift` reverts `InvalidT2Key` when `t2PubKey` is zero
- ✓ `lift` reverts `AmountIsZero` when amount is zero
- ✓ `lift` reverts `LiftFailed` if the balance does not strictly increase after the pull
- ✓ `lift` reverts `LiftLimitHit` if the bridge's post-lift balance exceeds `T2_TOKEN_LIMIT` (`uint128` max)
- ✓ `lift` pulls tokens via `safeTransferFrom` requiring prior approval
- ✓ `lift` credits the actual balance delta on T2, not the requested amount (handles fee-on-transfer)
- ✓ `lift` emits `LogLifted` with token, `t2PubKey` and the real lifted amount
- ✓ `permitLift` reverts when paused
- ✓ `permitLift` reverts `AddressBlocked` if `msg.sender` is sanctioned
- ✓ `permitLift` reverts `InvalidT2Key` when `t2PubKey` is zero
- ✓ `permitLift` forwards v/r/s to the token's permit which enforces the EIP-2612 deadline
- ✓ `permitLift` fails if the token does not implement permit
- ✓ `permitLift` credits the actual balance delta via `_lift`
- ✓ `permitLift` emits `LogLifted`
- ✓ `predictionMarketLift` reverts when paused
- ✓ `predictionMarketLift` reverts `AddressBlocked` if sender is sanctioned
- ✓ `predictionMarketLift` reverts `InvalidToken` for any token other than USDC or USDT
- ✓ `predictionMarketLift` derives the recipient from `msg.sender` via `deriveT2PublicKey`
- ✓ `predictionMarketLift` cannot misdirect funds to a third party (recipient is derived)



- ✓ `predictionMarketLift` emits `LogLiftedToPredictionMarket` with the derived T2 account
- ✓ `predictionMarketPermitLift` reverts `AddressBlocked` if sender is sanctioned
- ✓ `predictionMarketPermitLift` consumes the user's USDC permit before transferring
- ✓ `predictionMarketPermitLift` hardcodes the token to USDC
- ✓ `predictionMarketPermitLift` credits funds to the caller's own derived T2 account
- ✓ `predictionMarketPermitLift` emits `LogLiftedToPredictionMarket`
- ✓ `predictionMarketRecipientLift` reverts when paused
- ✓ `predictionMarketRecipientLift` reverts `AddressBlocked` if sender is sanctioned
- ✓ `predictionMarketRecipientLift` reverts `InvalidToken` for non-USDC, non-USDT tokens
- ✓ `predictionMarketRecipientLift` reverts `InvalidT2Key` when the supplied `t2PubKey` is zero
- ✓ `predictionMarketRecipientLift` credits the explicit T2 account in the parameter
- ✓ `predictionMarketRecipientLift` emits `LogLiftedToPredictionMarket`
- ✓ `claimLower` reverts when paused
- ✓ `claimLower` reverts `InvalidProof` on a malformed proof length
- ✓ `claimLower` reverts `AddressIsZero` when the recipient parsed from the proof is zero
- ✓ `claimLower` reverts `LowerIsUsed` if the `lowerId` was previously claimed or reverted
- ✓ `claimLower` reverts `BadConfirmations` if the lower's signatures do not reach quorum
- ✓ `claimLower` can be called by anyone (recipient does not have to be `msg.sender`)
- ✓ `claimLower` marks the `lowerId` used before the external token transfer
- ✓ `claimLower` transfers the full amount to the recipient via `safeTransfer`
- ✓ `claimLower` emits `LogLowerClaimed` with the `lowerId`
- ✓ `revertLower` reverts when paused
- ✓ `revertLower` reverts `InvalidProof` on a malformed proof
- ✓ `revertLower` reverts `PermissionDenied` when caller is neither recipient nor owner
- ✓ `revertLower` reverts `PermissionDenied` when the owner calls before 72 hours past `t2Timestamp`



- ✓ `revertLower` allows the recipient to revert at any time
- ✓ `revertLower` allows the owner to revert once `block.timestamp` exceeds `t2Timestamp + 3 days`
- ✓ `revertLower` reverts `LegacyLower` if `t2Sender` is zero (no refund target on T2)
- ✓ `revertLower` reverts `LowerIsUsed` if the lower was already claimed or reverted
- ✓ `revertLower` reverts `BadConfirmations` on missing quorum
- ✓ `revertLower` does not transfer any tokens on Ethereum (funds stay locked in bridge)
- ✓ `revertLower` marks the `lowerId` used so `claimLower` can no longer process it
- ✓ `revertLower` emits `LogLowerReverted` with token, `t2Sender`, recipient, amount, `lowerId`
- ✓ `registerRelayer` can only be called by the owner
- ✓ `registerRelayer` reverts `AddressIsZero` when relayer is the zero address
- ✓ `registerRelayer` reverts `RelayerAlreadyRegistered` when balance is non-zero
- ✓ `registerRelayer` sets `relayerBalance` to 1 as the registered-zero-fees sentinel
- ✓ `registerRelayer` emits `LogRelayerRegistered`
- ✓ `deregisterRelayer` can only be called by the owner
- ✓ `deregisterRelayer` reverts `RelayerNotRegistered` when balance is zero
- ✓ `deregisterRelayer` zeroes `relayerBalance` before transferring (CEI)
- ✓ `deregisterRelayer` transfers `(balance - 1)` USDC to the relayer when `balance > 1`
- ✓ `deregisterRelayer` makes no USDC transfer when balance equals 1 (no accrued fees)
- ✓ `deregisterRelayer` leaves the address re-registrable after the slot returns to zero
- ✓ `deregisterRelayer` emits `LogRelayerDeregistered`
- ✓ `relayerLift` reverts when paused
- ✓ `relayerLift` reverts `AddressBlocked` if the user (not `msg.sender`) is sanctioned
- ✓ `relayerLift` reverts `RelayerOnly` if `msg.sender` is not a registered relayer
- ✓ `relayerLift` reverts `AmountTooLow` if the computed `txCost` exceeds amount
- ✓ `relayerLift` calls `USDC.permit` with the user's signature and `deadline = type(uint256).max`



- ✓ relayerLift pulls the user's USDC via safeTransferFrom
- ✓ relayerLift deducts txCost from the credited amount inside an unchecked block
- ✓ relayerLift accrues txCost to the relayer's relayerBalance
- ✓ relayerLift triggers a USDC→ETH refund through _attemptRelayerRefund when triggerRefund is true
- ✓ relayerLift stores the new balance when triggerRefund is false
- ✓ relayerLift emits LogLiftedToPredictionMarket with the net (post-fee) amount
- ✓ relayerLower reverts when paused
- ✓ relayerLower reverts RelayerOnly if msg.sender is not a registered relayer
- ✓ relayerLower reverts InvalidToken when the proof token is not USDC
- ✓ relayerLower reverts AmountTooLow if txCost > amount
- ✓ relayerLower reverts LowerIsUsed for a replayed lowerId
- ✓ relayerLower reverts BadConfirmations on missing quorum
- ✓ relayerLower marks the lowerId used so the same lower can no longer be claimed
- ✓ relayerLower accrues txCost to the relayer and transfers (amount - txCost) to the user
- ✓ relayerLower triggers a USDC→ETH refund when triggerRefund is true
- ✓ relayerLower emits LogRelayerLowered with the lowerId and net amount
- ✓ _attemptRelayerRefund resets relayerBalance to 1 when the inner callback returns
- ✓ _attemptRelayerRefund preserves the prior balance on inner revert (catch branch)
- ✓ _attemptRelayerRefund emits LogRefundFailed only on the catch branch
- ✓ refundRelayerCallback reverts InvalidCaller unless msg.sender is address(this)
- ✓ refundRelayerCallback executes a USDC→WETH swap on the configured Uniswap V3 pool
- ✓ refundRelayerCallback reverts if the swap output falls below 98.7% of expected (slippage floor)
- ✓ refundRelayerCallback unwraps WETH to ETH via IWETH9.withdraw before forwarding
- ✓ refundRelayerCallback sends ETH to the relayer via a low-level call



- ✓ `uniswapV3SwapCallback` reverts `InvalidCaller` unless `msg.sender` is the configured pool
- ✓ `uniswapV3SwapCallback` pays `amount0Delta` USDC to the pool via `safeTransfer`
- ✓ `authorIsActive` returns true when the active bit is set for the id
- ✓ `authorIsActive` returns false for id 0 and any unregistered id
- ✓ `isAuthor` returns true when the registered bit is set for the id
- ✓ `isAuthor` returns false for id 0 and any never-registered id
- ✓ `isUsedLower` returns true when the `lowerId`'s bit is set in `usedLowers`
- ✓ `isUsedLower` returns false for any never-issued `lowerId`
- ✓ `corroborate` returns `TX_SUCCEEDED` when the `t2TxId` bit is set in `usedT2TxIds`
- ✓ `corroborate` returns `TX_FAILED` when `block.timestamp` exceeds expiry and the bit is unset
- ✓ `corroborate` returns `TX_PENDING` when the bit is unset and the expiry is in the future
- ✓ `checkLower` returns a zero-initialised struct on malformed proof length
- ✓ `checkLower` decodes the six lower fields and returns them
- ✓ `checkLower` reports `confirmationsRequired` equal to `numActiveAuthors/2 + 1`
- ✓ `checkLower` decrements `confirmationsProvided` for signatures whose author is not active
- ✓ `checkLower` decrements `confirmationsProvided` for duplicate signatures from the same active author
- ✓ `checkLower` sets `proofsValid` only when valid confirmations reach the required threshold
- ✓ `checkLower` reports `lowerIsUsed` from the `usedLowers` bitmap
- ✓ `confirmTransaction` walks the Merkle path hashing two nodes in sorted order at each step
- ✓ `confirmTransaction` returns true only if the computed root has been published
- ✓ `confirmTransaction` returns false for a leaf whose path resolves to an unknown root
- ✓ `confirmTransaction` with an empty `merklePath` checks `isPublishedRootHash(leafHash)` directly



- ✓ `name` returns the literal string "PredictorBridge"
- ✓ `deriveT2PublicKey` returns `keccak256(abi.encodePacked(t1Address))`
- ✓ `deriveT2PublicKey` is pure and deterministic
- ✓ `usdcEth` returns `latestAnswer / 1e6` (wei per USDC base unit)
- ✓ `_verifyConfirmations` in non-lower mode counts the sender as a free first confirmation if registered
- ✓ `_verifyConfirmations` in lower mode requires every confirmation to be an explicit signature
- ✓ `_verifyConfirmations` lazily activates a registered-but-benched author on first valid signature
- ✓ `_verifyConfirmations` recomputes the quorum after each lazy activation
- ✓ `_verifyConfirmations` deduplicates repeat signatures via the local `confirmedBitmap`
- ✓ `_verifyConfirmations` ignores signatures whose recovered address is not a registered author
- ✓ `_verifyConfirmations` returns immediately when valid count equals required (`== check`)
- ✓ `_verifyConfirmations` skips the `confirmedBitmap` write on the quorum-reaching iteration
- ✓ `_verifyConfirmations` reverts `BadConfirmations` if quorum is never reached
- ✓ `_recoverAuthorId` reads `r, s, v` at `offset + index*65` from `calldata`
- ✓ `_recoverAuthorId` normalises `v` from 0/1 to 27/28 if needed
- ✓ `_recoverAuthorId` enforces the low-s malleability bound on `s`
- ✓ `_recoverAuthorId` returns 0 for `v` outside {27,28} or for `s` above the low-s bound
- ✓ `_recoverAuthorId` returns 0 when `ecrecover` yields an address not registered as an author
- ✓ `_domainSeparator` binds signatures to NAME, version "1", `block.chainid` and `address(this)`
- ✓ `_toAddAuthorProofHash` hashes the dynamic `t1PubKey` bytes member, leaves `t2PubKey` raw (bytes32)



- ✓ `_toRemoveAuthorProofHash` hashes the dynamic `t1PubKey` member, leaves `t2PubKey` raw
- ✓ `_toPublishRootProofHash` encodes `rootHash`, `expiry`, `t2TxId` directly (all atomic)
- ✓ `_toLowerDataProofHash` encodes all six lower fields directly (all atomic)
- ✓ All four hashers wrap the struct hash with `EIP712_PREFIX (\x19\x01)` and `_domainSeparator`
- ✓ `_storeT2TxId` reverts `TxIdsUsed` if the bit is already set
- ✓ `_storeT2TxId` sets the bit so corroborate later reports `TX_SUCCEEDED`
- ✓ `_idToBitmap` returns `(id >> 8, 1 << (id & 255))`
- ✓ `_idToBitmap` shift amount is always `< 256` so it cannot overflow
- ✓ `_authorBit` returns `1 << id` and is unsafe for `id ≥ 256` (bounded by `MAX_AUTHORS`)
- ✓ `_isBitSet` returns true iff `(bitmap & bit)` is non-zero
- ✓ `_toAddress` returns `address(uint160(uint256(keccak256(t1PubKey))))`
- ✓ `_isPredictionMarketToken` returns true only for USDC or USDT
- ✓ `_isCorrectLength` rejects proofs shorter than `MINIMUM_LOWER_PROOF_LENGTH` (246 bytes)
- ✓ `_isCorrectLength` rejects proofs whose signature tail length is not a multiple of 65
- ✓ `_extractLowerData` reverts `InvalidProof` if length is wrong
- ✓ `_extractLowerData` reads token (20 bytes at offset 0) with `shr(96)`
- ✓ `_extractLowerData` reads amount (32 bytes at offset 20) directly
- ✓ `_extractLowerData` reads recipient (20 bytes at offset 52) with `shr(96)`
- ✓ `_extractLowerData` reads `lowerId` (4 bytes at offset 72) with `shr(224)`
- ✓ `_extractLowerData` reads `t2Sender` (32 bytes at offset 76) directly
- ✓ `_extractLowerData` reads `t2Timestamp` (8 bytes at offset 108) with `shr(192)`
- ✓ `_extractLowerData` produces clean high bits for every short field via `shr`
- ✓ `_processLower` reverts `LowerIsUsed` when the `lowerId` bit is set in `usedLowers`



- ✓ `_processLower` marks the `lowerId` used BEFORE verifying confirmations (fast-fail then commit)
- ✓ `_processLower` rebuilds the EIP-712 digest from the supplied six fields
- ✓ `_processLower` invokes `_verifyConfirmations` in lower mode over `lowerProof[116:]`
- ✓ `_lift` reverts `AmountIsZero` when amount is zero
- ✓ `_lift` measures the bridge balance before and after the transfer
- ✓ `_lift` reverts `LiftFailed` if `newBalance` is not strictly greater than `existingBalance`
- ✓ `_lift` reverts `LiftLimitHit` if `newBalance` exceeds `T2_TOKEN_LIMIT (type(uint128).max)`
- ✓ `_lift` returns the actual balance delta as `liftedAmount`
- ✓ `_addNewAuthor` reverts `TooManyAuthors` when `nextAuthorId` is 256 or more
- ✓ `_addNewAuthor` reverts `T2KeyInUse` if the `t2PubKey` is already bound
- ✓ `_addNewAuthor` writes both `id→address/key` and `address/key→id` mappings
- ✓ `_addNewAuthor` sets the `isAuthor` bit but NOT the active bit
- ✓ `_activateAuthor` sets the active bit and increments `numActiveAuthors`
- ✓ `_setAuthor` flips the `isAuthor` bit on
- ✓ `_clearAuthor` flips the `isAuthor` bit off
- ✓ `_clearAuthorActive` flips the active bit off
- ✓ `_initialiseAuthors` immediately activates each seed author via `_activateAuthor`
- ✗ `permitLift-permit()` can be front-run causing DoS
- ✗ `permitLift-permit()` can be front-run causing DoS



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Chainalysis Sanctions Oracle (IChainalysis)	External Oracle	checkAddress modifier calls isSanctioned() on lift paths	Oracle is live, honest, and responsive	Oracle downtime/ compromise bypasses or blocks all lifts; sanctions check not applied on lower/claim paths
Chainlink USDC/ETH Feed (IChainlinkV3 Aggregator)	Price Oracle	usdcEth() uses latestAnswer() to price gas reimbursement for relayers	Feed is fresh, non-stale, correct decimals	No staleness check, no min/ max bounds check; stale or manipulated price causes relayer over/ underpayment
Uniswap V3 USDC/WETH Pool (IUniswapV3Pool)	DEX / Liquidity Venue	refundRelayerCall back() swaps USDC → WETH for relayer ETH refund	Pool is liquid and not manipulated at swap time	hardcoded 1.3% slippage tolerance may be insufficient
WETH9 (IWETH9)	Token Wrapper	withdraw() called after Uniswap swap to unwrap WETH → ETH	Contract behaves as standard WETH9	-



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
USDC (IERC20 + IERC20Permit)	External ERC20	Primary bridge token; used in lift, lower, relayer flows	Standard ERC20Permit, 6 decimals, no fee-on-transfer	Permit front-running invalidates user signature; USDC blacklist can break transfers mid-flow
USDT (IERC20)	External ERC20	Accepted for prediction market lifts only	Standard ERC20, 6 decimals	USDT has no permit; non-standard approval pattern; no return value on transfer (SafeERC20 handles)
OpenZeppelin SafeERC20	Library	Wraps all ERC20 transfers with revert-on-failure	Library is correct and up to date	Bug in library could cause silent failures (very low risk on audited OZ versions)
OpenZeppelin PausableUpgradable	Base Contract	Owner can pause all state-changing ops	Pause is used responsibly	Malicious/compromised owner can freeze funds indefinitely
OpenZeppelin UUPSUpgradable	Base Contract	Owner can upgrade implementation	Upgrade authority is secured; new impl is safe	Malicious upgrade wipes all state or steals funds
ecrecover (EVM built-in)	Cryptographic Primitive	Recovers author T1 address from confirmation signatures	Signature malleability handled	s value upper-bound check present; v normalization present-looks handled

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
PredictorBridge.sol	constructor()	What this function can do Set 6 immutable addresses; disable initializers on implementation What this function cannot/should not do Called again; write proxy storage; move tokens Main invariant(s) All immutables non-zero; implementation locked forever Access level Once at deployment implicit restriction
PredictorBridge.sol	initialize()	What this function can do Set owner; init OZ modules; seed initial author set What this function cannot/should not do Called twice; accept < MIN_AUTHORS; accept mismatched key arrays Main invariant(s) numActiveAuthors ≥ 4; all T1$\leftrightarrow$T2 mappings consistent; nextAuthorId = 1 Access level Once-initializer modifier; no caller restriction -front-run risk if not deployed atomically



Contract Name	Function	Category
PredictorBridge.sol	lift()	What this function can do Pull any token from caller into bridge; emit LogLifted with actual received amount What this function cannot/should not do Lift zero amount; lift to zero T2 key; exceed T2_TOKEN_LIMIT; lift from sanctioned address Main invariant(s) newBalance > existingBalance; bridge balance ≤ T2_TOKEN_LIMIT Access level Public-whenNotPaused, nonReentrant, checkAddress(msg.sender)
PredictorBridge.sol	permitLift()	What this function can do Execute ERC-2612 permit then lift tokens in one tx; emit LogLifted What this function cannot/should not do Lift zero; lift to zero T2 key; lift from sanctioned address; work if permit already consumed Main invariant(s) Permit consumed atomically with transfer; same balance invariant as lift() Access level Public-whenNotPaused, nonReentrant, checkAddress(msg.sender) -permit front-run invalidates signature and reverts entire tx

Contract Name	Function	Category
PredictorBridge.sol	predictionMarketLift()	What this function can do Lift USDC or USDT only to caller's derived T2 key What this function cannot/should not do Accept any token other than USDC/USDT; lift zero; lift from sanctioned address Main invariant(s) T2 key always = keccak256(msg.sender); only USDC/USDT accepted Access level Public-whenNotPaused, nonReentrant, checkAddress(msg.sender)
PredictorBridge.sol	predictionMarketPermitLift	What this function can do Permit + lift USDC only to caller's derived T2 key in one tx What this function cannot/should not do Accept USDT; lift from sanctioned address; work if permit already consumed Main invariant(s) T2 key always = keccak256(msg.sender); only USDC permitted Access level Public-whenNotPaused, nonReentrant, checkAddress(msg.sender) -same permit front-run risk as permitLift()

Contract Name	Function	Category
PredictorBridge.sol	predictionMarketRecipientLift()	What this function can do Lift USDC or USDT to any specified non-zero T2 key on behalf of caller What this function cannot/should not do Accept non-USDC/USDT; lift to zero T2 key; lift from sanctioned address Main invariant(s) T2 key must be non-zero; only USDC/USDT accepted; balance invariant holds Access level Public-whenNotPaused, nonReentrant, checkAddress(msg.sender)
PredictorBridge.sol	relayerLift()	What this function can do Registered relayer lifts USDC for user; deducts tx cost from amount; optionally triggers ETH refund The relayer controls the gasCost parameter. A permit signed by the user never expires. If a relayer goes rogue or is compromised later What this function cannot/should not do Called by non-relayer; lift if txCost > amount; lift from sanctioned user; lift non-USDC Main invariant(s) relayerBalance always ≥ 1 if registered; amount sent to T2 = amount - txCost; txCost never exceeds amount Access level Registered relayer only-whenNotPaused, nonReentrant, checkAddress(user) -no minimum amount check; oracle staleness inflates txCost; permit front-run risk

Contract Name	Function	Category
PredictorBridge.sol	revertLower()	What this function can do Recipient reverts their own lower anytime; owner reverts after 72h on behalf of recipient; marks lowerId used What this function cannot/should not do Called by non-recipient before 72h; revert legacy lower (zero t2Sender); revert already-used lowerId Main invariant(s) Only recipient or owner (after delay) can revert; lowerId marked used atomically; t2Sender must be non-zero Access level Recipient anytime; Owner after OWNER_REVERT_LOWER_DELAY (3 days)-whenNotPaused, nonReentrant
PredictorBridge.sol	addAuthor()	What this function can do Add new author or reactivate existing one; permanently link T1↔T2 keys; store t2TxId What this function cannot/should not do Add already-active author; change T2 key of existing author; exceed MAX_AUTHORS (255); accept expired request Main invariant(s) T1↔T2 mapping is permanent and immutable once set; t2TxId never reused; expiry enforced Access level Public-whenNotPaused, withinCallWindow(expiry) -quorum of existing authors must confirm

Contract Name	Function	Category
PredictorBridge.sol	removeAuthor() 	What this function can do Remove active author; decrement numActiveAuthors; store t2TxId What this function cannot/should not do Remove non-author; reduce authors below MIN_AUTHORS (4); accept expired request; reuse t2TxId Main invariant(s) numActiveAuthors always ≥ 4; removed author loses signing authority immediately Access level Public-whenNotPaused, withinCallWindow(expiry) -quorum of existing authors must confirm
PredictorBridge.sol	publishRoot()	What this function can do Publish a confirmed T2 Merkle root to Ethereum; store rootHash; store t2TxId What this function cannot/should not do Publish same rootHash twice; accept expired request; publish without quorum confirmations Main invariant(s) Each rootHash published exactly once; t2TxId never reused; expiry enforced Access level Public-whenNotPaused, withinCallWindow(expiry) -quorum of authors must confirm



Contract Name	Function	Category
PredictorBridge.sol	registerRelayer()	What this function can do Owner registers a new relayer; sets relayerBalance to 1 as sentinel What this function cannot/should not do Register zero address; register already-registered relayer; called by non-owner Main invariant(s) relayerBalance = 1 signals registered but empty; never 0 for active relayer Access level onlyOwner
PredictorBridge.sol	deregisterRelayer()	What this function can do Owner deregisters relayer; returns any accumulated USDC balance (balance - 1) to relayer What this function cannot/should not do Deregister unregistered relayer; called by non-owner; lose relayer funds Main invariant(s) relayerBalance set to 0 after deregister; USDC returned = balance - 1 (sentinel subtracted) Access level onlyOwner, nonReentrant



Contract Name	Function	Category
PredictorBridge.sol	pause()	What this function can do Owner pauses all state-changing bridge operations What this function cannot/should not do Called when already paused; called by non-owner Main invariant(s) whenNotPaused functions revert while paused; owner can always unpause Access level onlyOwner, whenNotPaused
PredictorBridge.sol	unpause()	What this function can do Owner unpauses all state-changing bridge operations What this function cannot/should not do Called when not paused; called by non-owne Main invariant(s) All paused functions resume after unpause Access level onlyOwner, whenPaused

Contract Name	Function	Category
PredictorBridge.sol	refundRelayerCallback()	What this function can do Swap accumulated USDC → WETH via Uniswap; unwrap to ETH; send ETH to relayer What this function cannot/should not do Called by anyone except address(this); proceed if slippage > 1.3%; silently fail ETH send Main invariant(s) Only self-callable; ETH received ≥ 98.7% of expected; relayerBalance reset to 1 on success Access level Internal callback-msg.sender == address(this) only -hardcoded 1.3% slippage may be insufficient; ETH send failure silently ignored via pop(success); stale oracle inflates minimum check
PredictorBridge.sol	uniswapV3SwapCallback	What this function can do Pay USDC owed to Uniswap pool during swap callback What this function cannot/should not do Called by anyone except configured Uniswap pool; pay wrong token or wrong amount Main invariant(s) Only pool can call; amountODelta always positive (USDC owed); amount paid = amountODelta exactly Access level msg.sender == UNISWAP_V3_USDC_WETH_POOL only



3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
PredictorBridge.sol	ERC20 (USDC)	User lifted USDC pending T2 settlement; relayer accumulated fee balance (relayerBalance mapping)
PredictorBridge.sol	ERC20 (USDT)	User lifted USDT pending T2 settlement
PredictorBridge.sol	ERC20 (Any Token)	Any token lifted via generic lift() or permitLift() pending T2 settlement
PredictorBridge.sol	ETH (Native)	Temporarily holds ETH received from WETH unwrap during relayer refund flow via receive()
PredictorBridge.sol	WETH (ERC20)	Transiently holds WETH received from Uniswap swap before unwrapping to ETH in refundRelayerCallback()



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
PredictorBridge.sol	lift()	Any ERC20 (from user)	–	Public	–
PredictorBridge.sol	permitLift()	Any ERC20 (from user)	–	Public	Permit front-run invalidates signature and reverts tx; same token risk as lift()
PredictorBridge.sol	predictionMarketLift()	USDC or USDT (from user)	–	Public	USDT non-standard approval; no minimum amount check
PredictorBridge.sol	predictionMarketPermitLift()	USDC only (from user)	–	Public	Permit front-run risk; hardcoded to USDC only
PredictorBridge.sol	predictionMarketRecipientLift()	USDC or USDT (from user)	–	Public	Caller lifts but T2 recipient is arbitrary; no sanctions check on T2 recipient
PredictorBridge.sol	relayerLift()	USDC (from user via permit)	–	Registered Relayer	Permit front-run; stale oracle inflates txCost; no minimum bridged amount; relayer supplies gasCost unchecked
PredictorBridge.sol	claimLower()	–	Any ERC20 (to recipient)	Public	No sanctions check on recipient; sanctioned address freely withdraws



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
PredictorBridge.sol	relayerLower()	–	USDC (to user minus txCost)	Registered Relayer	No sanctions check on recipient; stale oracle inflates txCost; relayer supplies gasCost unchecked
PredictorBridge.sol	revertLower()	–	– (funds stay in bridge)	Recipient / Owner	Funds locked in bridge after revert-no immediate return to T2; owner can revert anyone's lower after 3 days
PredictorBridge.sol	deregisterRelayer()	–	USDC (to relayer)	Owner only	Owner can drain relayer balance at will; no relayer consent required
PredictorBridge.sol	refundRelayerCallback()	USDC (internal relayer fees)	ETH (to relayer)	Self only (address (this))	Hardcoded 1.3% slippage insufficient in volatile markets; ETH send failure silently ignored via pop(success); stale Chainlink oracle skews minimum ETH check
PredictorBridge.sol	uniswapV3SwapCallback()	–	USDC (to Uniswap pool)	Uniswap pool only	Spoofed pool address would drain USDC-mitigated by hardcoded pool address check



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
PredictorBridge.sol	receive()	ETH (from WETH unwrap)	–	Anyone	Accepts ETH from any sender; ETH sent directly is permanently locked with no withdrawal path

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of The Truth Network. We performed our audit according to the procedure described above.

No critical issues in The Truth Network, just 3 issues of Medium severity were found, which have been noted and resolved by the Truth Network team.

All the issues were resolved.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

June 2026

For



Predictor™



QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com